

TEXT EMBEDDINGS (METİN YERLEŐTİRMELERİ) ÇALIŐMA RAPORU

DeepLearning.AI & Google Cloud 'Understanding and Applying Text Embeddings' Kursu Teknik Uygulama ve Kod Rehberi

Metin Yerleőtirmeleri (Embeddings) Nedir?

Metin yerleőtirmeleri (text embeddings), kelimelerin, cümlelerin veya paragrafların bilgisayarlar tarafından anlaşılabilmesi için yüksek boyutlu sayısal vektörlere dönüőtürülmesidir. Bu süreçte en kritik nokta, **semantik (anlamsal) benzerliğin** matematiksel olarak korunmasıdır. Birbirine anlamsal olarak yakın olan metinler (örneğin 'kedi' ve 'köpek' veya 'yapay zeka' ve 'derin öğrenme'), bu yüksek boyutlu uzayda birbirine yakın koordinatlara sahip olur. Bu rehber, DeepLearning.AI platformunda Andrew Ng ve Nikita Namjoshi tarafından verilen eğitimin pratik kod uygulamalarını ve gerçek dünya senaryolarını detaylandırmak amacıyla hazırlanmıştır.

Günlük Hayattan Analoji: Kütüphane Koordinat Sistemi

Kütüphane Analjisi: Büyük bir kütüphaneyi hayal edin. Eğer kitapları rastgele dizerseniz, istediğiniz kitabı bulmak saatler alır. Ancak akıllı bir kütüphaneci, her kitaba 3 boyutlu bir koordinat verir: *Raf Numarası (X)*, *Sütun Numarası (Y)*, *Yükseklik (Z)*. Bilim kurgu kitapları kütüphanenin sağ üst köşesindeki raflarda (birbirine yakın koordinatlarda) toplanırken, yemek tarifleri sol alt köşede yer alır. İşte Embeddings, kelimelere ve cümlelere 3 boyut değil, tam **768 boyutlu (veya daha fazla)** birer koordinat atayarak onları anlamsal özelliklerine göre kütüphanedeki gibi akıllıca gruplar.

BÖLÜM 1: Vertex AI Embeddings API ve Temel Kullanım

Google Cloud'un Vertex AI platformu, dünya genelinde kullanılan gelişmiş metin yerleőtirme modellerini barındırır. Kurs kapsamında kullanılan temel model **textembedding-gecko@001** (veya daha güncel projeler için **text-embedding-005**) modelidir. Bu model, her metin girdisi için **768 boyutlu** bir yoğun vektör üretir.

Python Entegrasyon Kodu:

```
# Vertex AI SDK Başlatma ve Temel Metin Yerleőtirme Oluőturma
import vertexai from vertexai.language_models
import TextEmbeddingModel, TextEmbeddingInput # Google Cloud projenizi ve konumunuzu tanımlayın
PROJECT_ID = "gcp-projeniz-123"
LOCATION = "us-central1"
vertexai.init(project=PROJECT_ID, location=LOCATION) # Önceden eğitilmiş Text Embedding modelini yükleyin
model = TextEmbeddingModel.from_pretrained("textembedding-gecko@001") # Tek bir metin için yerleőtirme (embedding)
alma text_to_embed = "Yapay zeka teknolojileri iş süreçlerini otomatikleőtiriyor."
embeddings = model.get_embeddings([text_to_embed]) # Elde edilen ilk vektörü ve özelliklerini inceleyin
vector = embeddings[0].values
print(f"Vektör Tipi: {type(vector)}")
print(f"Vektör Boyutu (Dimension): {len(vector)}") # Çıktı: 768
print(f"İlk 5 Sayısal Değer: {vector[:5]}")
```

Google Cloud Vertex AI, yerleőtirmeleri daha spesifik görevler için optimize etmek amacıyla **task_type** parametresini destekler. Aşağıdaki tabloda yaygın task tipleri listelenmiştir:

Görev Tipi (Task Type)	Açıklama	En Uygun Kullanım Senaryosu
RETRIEVAL_DOCUMENT	Büyük dökümanların veya paragrafların anlamsal temsili için optimize edilmiştir.	RAG veritabanındaki kaynak dökümanlar.
RETRIEVAL_QUERY	Kısa kullanıcı sorguları için optimize edilmiş hafif yerleőtirmeler üretir.	Arama çubuğuna yazılan anlık sorular.
CLASSIFICATION	Sınıflandırma algoritmalarına girdi oluşturmak üzere tasarlanmıştır.	Duygu analizi, spam filtresi modelleri.
CLUSTERING	Verilerin gözetimsiz olarak gruplandırılmasında en yüksek doğruluğu sağlar.	Müşteri taleplerini otomatik konulara ayırma.

BÖLÜM 2: Vektör Uzayının Boyutunu İndirme ve Görselleőtirme

768 boyutlu bir uzayı insan gözüyle hayal etmek ve görselleőtirmek imkansızdır. Bu engeli aşmak için, veri biliminde **PCA (Temel Bileşenler Analizi)** veya **t-SNE** gibi boyut indirgeme algoritmaları kullanılır. Bu algoritmalar, yüksek boyutlu uzaydaki varyansı ve ilişkileri koruyarak veriyi 2 ya da 3 boyuta indirger.

Günlük Hayattan Analoji: Dünya Küresini Haritaya İndirmek

Harita Analojisi: 3 boyutlu küre şeklindeki Dünya'yı düz bir kağıda (2 boyutlu harita) aktarmaya benzer. Ülkelerin birbirine göre yönleri ve sınırları korunur, ancak boyutlarında kaçınılmaz olarak ufak bozulmalar yaşanır. PCA algoritması da tam olarak bunu yapar; 768 boyutlu 'anlam küresini' 2 boyutlu bir düzleme iz düşürür.

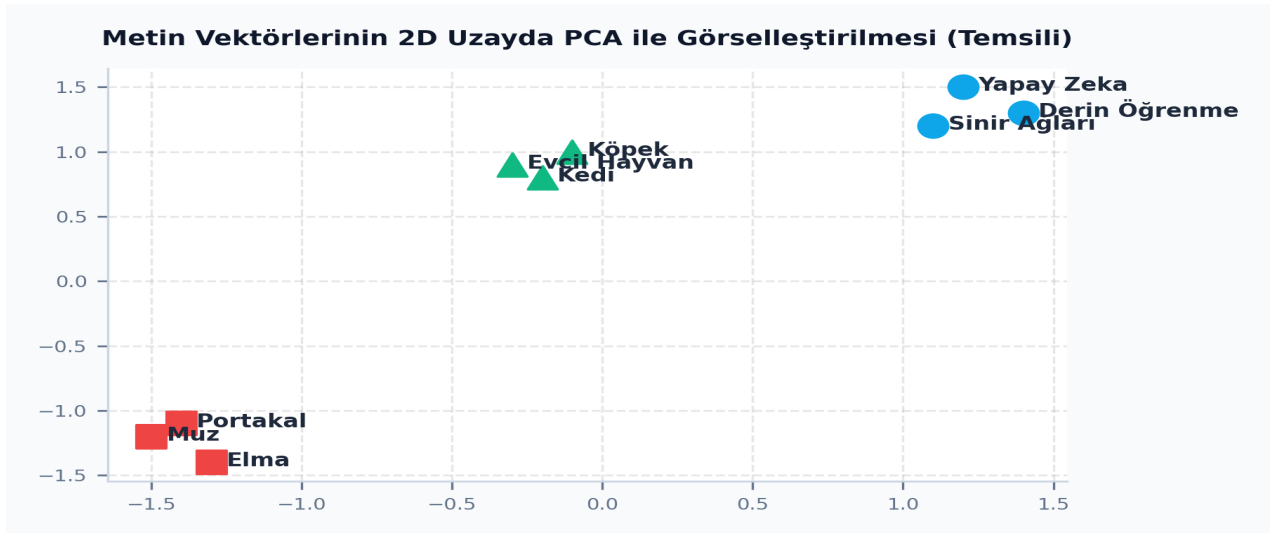
Boyut İndirgeme ve Grafik Hazırlama Kodu:

```
# PCA Kullanarak Yerleőtirmeleri 2 Boyuta Düşürme ve Çizdirme
import numpy as np
import pandas as pd
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt

# 768 boyutlu 3 adet embedding vektörümüz olduğunu varsayalım (Gereksinim: np.ndarray)
embeddings_matrix = np.random.rand(9, 768)

# Temsili veri # PCA modelini tanımlayın (2 temel bileşene indirgeyeceğiz)
pca = PCA(n_components=2)
reduced_vectors = pca.fit_transform(embeddings_matrix)

# Elde edilen 2D koordinatların dataframe'e aktarılması
df_2d = pd.DataFrame(reduced_vectors, columns=['x', 'y'])
print(df_2d.head())
```



Şekil 1: Kelimelerin semantik gruplarının PCA ile 2D düzleme izdüşümü.

BÖLÜM 3: Yerleőtirmelerin Pratik İőletme Uygulamaları

Yerleőtirmeler (Embeddings), bir kez elde edildikten sonra geleneksel makine öğrenmesi algoritmalarına girdi (feature) olarak beslenebilir. Bu sayede karmaşık metin problemleri, basit sayısal analizlere dönüşür. Eğitimde öne çıkan üç ana senaryo şunlardır: **Sınıflandırma**, **Kümeleme** ve **Aykırı Değer Tespiti**.

1. Sınıflandırma (Classification):

Müşteri yorumlarının duygu analizi (olumlu/olumsuz) gibi etiketli verilerde, yerleőtirme vektörleri girdi özellikleri olarak kullanılır ve üzerlerine bir **Logistic Regression** modeli eğitilir.

```
from sklearn.linear_model import LogisticRegression from sklearn.model_selection import train_test_split #
X: Girdilerin embedding vektörleri (N, 768), y: Sınıf Etiketleri (0: Negatif, 1: Pozitif) X_train, X_test,
y_train, y_test = train_test_split(embeddings_list, labels, test_size=0.2) # Sınıflandırıcıyı eğitin
classifier = LogisticRegression() classifier.fit(X_train, y_train) # Başarı skorunu ölçün accuracy =
classifier.score(X_test, y_test) print(f"Duygu Analizi Sınıflandırma Başarısı: %{accuracy * 100:.2f}")
```

2. Kümeleme (Clustering):

Verimizde önceden tanımlanmış bir etiket yoksa, anlamsal olarak benzer metinleri otomatik gruplamak için **K-Means** algoritması tercih edilir. Örneğin, gelen binlerce müşteri şikayetini otomatik olarak konularına ayırmak.

```
from sklearn.cluster import KMeans # 3 farklı ana grup (küme) oluşturmak isteyelim num_clusters = 3 kmeans =
KMeans(n_clusters=num_clusters, random_state=42) kmeans.fit(embeddings_list) # Her metne atanan küme ID'sini
alın cluster_assignments = kmeans.labels_ for i, text in enumerate(sample_texts): print(f"Metin: '{text}' ->
Küme No: {cluster_assignments[i]}")
```

3. Aykırı Değer Tespiti (Outlier Detection):

Bir metin grubuna anlamsal olarak uymayan, konunun tamamen dışındaki girdileri bulma işlemidir. Gruptaki tüm yerleőtirmelerin ortalaması (centroid) alınır, her bir cümle için bu merkeze olan **Cosine Uzaklığı** hesaplanır. Ortalamaya en uzak olan cümle aykırı değer olarak etiketlenir.

```
from sklearn.metrics.pairwise import cosine_similarity # 1. Adım: Tüm vektörlerin ortalamasını (küme
merkezini) hesaplayın centroid = np.mean(embeddings_list, axis=0) # 2. Adım: Her vektörün merkeze olan Cosine
Benzerliğini bulun similarities = cosine_similarity(embeddings_list, [centroid]) # 3. Adım: Benzerliği en
düşük (uzaklığı en yüksek) olanı tespit edin outlier_index = np.argmin(similarities) print(f"Gruptan sapan
aykırı cümle: '{sample_texts[outlier_index]}")
```

BÖLÜM 4: Vertex AI ile Metin Üretimi ve LLM Parametre Kontrolü

Büyük Dil Modelleri (LLM), kelimelerin olasılık dağılımlarını hesaplayarak sonraki kelimeleri üretir. Sadece yerleőtirmeleri okumakla kalmaz, aynı zamanda çıktı kalitesini kontrol etmek için kritik parametreleri ayarlayabiliriz. Bu parametreler **Sıcaklık (Temperature)**, **Top-K** ve **Top-P**'dir.

Günlük Hayattan Analoji: Yazarın Kahve Tercihi

Yazar ve Kahve Analjisi: Bir yazara makale yazdırıyorsunuz. • **Sıcaklık = 0 (Kahvesiz/Sade):** Yazar son derece ciddi, mantıklı ve her seferinde kelimesi kelimesine aynı, garantici cümleyi yazar (Deterministic). • **Sıcaklık = 1 (Çift Espresso):** Yazarın hayal gücü tavan yapar. Daha riskli, beklenmedik ve yaratıcı kelimeler seçer (Creative / Random).

Dil Modeli Yapılandırma ve Çağrı Kodu:

```
# Vertex AI ile LLM Parametrelerinin Ayarlanması ve Metin Üretimi
from vertexai.generative_models import GenerativeModel, GenerationConfig
# Modeli başlatın
model = GenerativeModel("gemini-1.5-flash")
# Üretim parametrelerini yapılandırın
generation_config = GenerationConfig(temperature=0.2, # Düşük değer: Daha tutarlı ve mantıklı çıktılar
top_p=0.85, # Nucleus Sampling: Kümülatif olasılığı %85 olan kelimeler
top_k=40, # En yüksek olasılıklı ilk 40 kelimeyi sınırla
max_output_tokens=256 # Maksimum yanıt uzunluğu )
prompt = "Metin yerleőtirmelerinin veri bilimindeki önemini 2 cümleyle açıkla."
response = model.generate_content(prompt, generation_config=generation_config)
print(response.text)
```

BÖLÜM 5: Semantik Arama ve Soru-Cevap (Q&A;) Sistemi (RAG)

Bilgi Geri Kazanımı (Information Retrieval) süreçlerinde, binlerce döküman içinden en alakalı bilgiyi bulmak temel zorluktur. Geleneksel anahtar kelime aramaları (SQL LIKE veya Elasticsearch BM25) eş anlamlı kelimeleri ve bağlamı kaçıır. **Semantik Arama** ise sorunun kelimelerine değil, anlamına odaklanır. Google'ın açık kaynaklı **ScaNN (Scalable Nearest Neighbors)** kütüphanesi, milyonlarca vektör arasında milisaniyeler içinde arama yapabilen 'Yaklaşık En Yakın Komşu' (ANN) indeksleme altyapısı sunar.

Günlük Hayattan Analoji: Süpermarket Fihristi

Süpermarket Fihristi Analjisi: Süpermarkete girip 'sabun' arıyorsunuz. Anahtar kelime araması sadece üzerinde 'sabun' yazan reyonlara bakar. Ancak semantik arama, 'hijyen', 'temizlik', 'duş jeli' gibi kelimeleri de anlayarak sizi doğrudan kişisel bakım reyonuna yönlendirir. ScaNN ise reyonları önceden gruplayarak (clustering), gıda reyonlarının tamamını aramadan doğrudan temizlik reyonuna odaklanmanızı sağlar, böylece tüm marketi gezmek zorunda kalmazsınız.

ScaNN İndeksi Kurulumu ve RAG Akışı:

Aőağıdaki kod bloğunda, döküman yerleőtirmeleri üzerinden bir ScaNN indeksi oluşturulması, kullanıcı sorgusuna en yakın dökümanın hızlıca bulunması ve bu dökümanın dil modeline (RAG) bağlam (context) olarak verilerek doğru cevabın üretilmesi gösterilmiştir:

```
# ScaNN İndeksi ve RAG (Soru-Cevap) Uygulaması import scann import numpy as np from vertexai.generative_models import GenerativeModel # 1. Adım: Veritabanındaki dökümanların embedding matrisini hazırlayın # doc_embeddings.shape -> (1000, 768) doc_embeddings = np.array(embeddings_list) # 2. Adım: ScaNN Arama Motorunu Yapılandırın (Dot Product Mesafesiyle) searcher = scann.scann_ops_pybind.builder(doc_embeddings, num_neighbors=1, distance_measure="dot_product") \ .tree(num_leaves=10, num_leaves_to_search=5, training_sample_size=200) \ .score_ah(2, anisotropic_quantization_threshold=0.2) \ .reorder(10) \ .build() # 3. Adım: Kullanıcı sorusunu vektöre dönüőtürün ve arayın query = "Vertex AI ile nasıl soru-cevap sistemi kurulur?" query_embedding = model.get_embeddings([query])[0].values # En yakın dökümanı ScaNN ile bulun (Milisaniyeler sürer) neighbors, distances = searcher.search(query_embedding) retrieved_doc_index = neighbors[0] best_context = corpus_texts[retrieved_doc_index] # 4. Adım: Bulunan dökümanı LLM'e prompt (bağlam) olarak gönderin prompt = f""" Aőağıdaki bağlamı kullanarak soruyu yanıtla. Bilgin dışındaysa uydurma. Bağlam: {best_context} Soru: {query} Yanıt: """ llm = GenerativeModel("gemini-1.5-flash") qa_response = llm.generate_content(prompt) print("Sistem Yanıtı:\n", qa_response.text)
```

Sonuç Değerlendirmesi

Metin yerleőtirmeleri, modern üretken yapay zeka ve arama sistemlerinin görünmez kahramanlarıdır. Bu çalışma raporunda gösterilen yöntemler sayesinde, işletmeler sadece kelime eşleřmelerine dayalı eski arama motorları yerine, kullanıcı niyetini anlayan akıllı sistemler (RAG) kurabilirler. Google Cloud'un Vertex AI platformu ve ScaNN kütüphanesi, bu gelişmiş modelleri ölçeklenebilir ve yüksek performanslı bir şekilde üretime (production) taşımak için en güçlü araç takımlarından birini sunmaktadır.

Kaynaklar ve Eğitim Referansları

- **Kurs:** Understanding and Applying Text Embeddings (DeepLearning.AI)
- **Eğitmenler:** Andrew Ng (DeepLearning.AI Kurucusu), Nikita Namjoshi (Google Cloud Developer Advocate)
- **Modeller:** Google Vertex AI Text-Embeddings API (textembedding-gecko@001, text-embedding-005)
- **Kütüphaneler:** Google ScaNN (Scalable Nearest Neighbors), Scikit-Learn, Matplotlib, NumPy, Pandas